

①

①

Page No.	
Date	

ch. 1: - Collections

* Introduction to Collection Framework: →

→ A collection is an object that groups multiple objects into a single unit.

→ It is also called as Container.

→ Collections are used to store, retrieve, manipulate and communicate aggregate data.

→ A Java Collection Framework (JCF) is a unified architecture for representing and manipulating collections.

→ Using Java collection frameworks we can perform operations on a set of objects such as searching, sorting, insertion, manipulation and deletion.

→ The framework contains the following

* 1. Interfaces: → These are abstract data types that represent collections. Interfaces allow collections to be manipulated independently of the details of their representation.
e.g., List, Set, Map etc.

* 2. Implementations: → These are the classes that implement the collection interfaces. They are reusable data structures.

e.g., ArrayList, LinkedList classes that implement the List interface.

* 3. Algorithms: → These are the methods that perform useful computations.

(2)

(11)

Page No.	
Date	

such as searching and sorting on collection objects.

These algorithms are defined as static methods within the collection classes.

e.g. `BinarySearch()` `sort()` `max()` `min()`

* Advantages of collections.

1. Reduces programming efforts :- By providing useful data structures and algorithms JCF reduces the prod programming efforts.

2. Increases program speed and quality
* → It provides high performance, high quality implementation of useful data structure and algorithms.

3. Allows interoperability among unrelated API's * → All collections interfaces are the medium by which API's send collections to one another.

4. Reduce efforts to learn and to use new API's * →

All collections interfaces and classes form a hierarchy. Hence the common methods are inherited by most collections.

5. Reduces effort to design new API's * → The use of inheritance and interfaces in the collection framework makes it possible to add new features to an existing collections or design a new one easily.

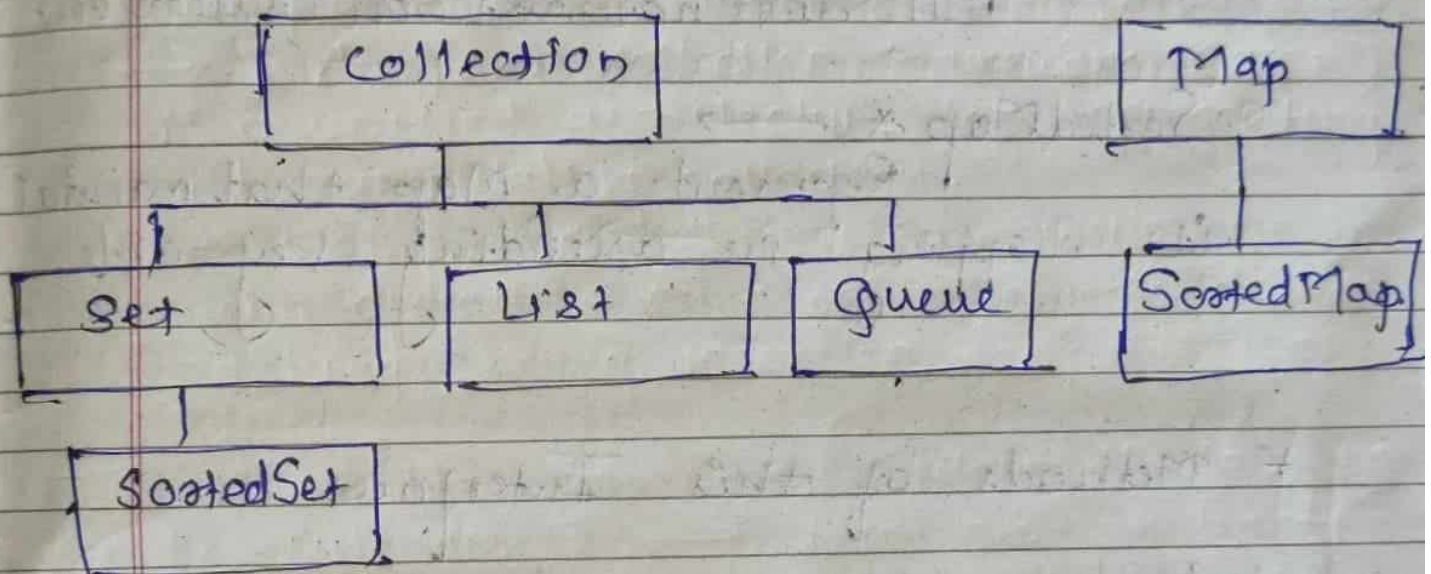
(8)

(11)

Page No.	
Date	

6. Allows for software reuse: $\rightarrow$
 New data structures and new algorithms built on standard Collection interfaces are reusable.

* Collection Interfaces and Classes
 * $\rightarrow$



* 1. Collection *! $\rightarrow$

It is the most general Collection interface type.
 It represents a group of objects known as elements.

* 2. List *! $\rightarrow$

It represents an ordered collection. List can contain duplicate elements.

* 3. Set *! $\rightarrow$ It

It represents an unordered collection that does not permit duplicate elements.

(2)

(IV)

Page No.			
Date			

5. SortedSet *: →

It represents a set whose elements are ordered in a sorted order.

6. Map *: →

Represents key-value pairs. A map can not contain duplicate keys. Each key can map to at most one value.

It does not extend Collection.

7. SortedMap *: →

Represents a Map that maintains its mapping in ascending key order.

* Methods of this Interfaces are:

1. boolean add(Object obj) → Adds obj to a collection. Returns true if obj is added to the collection. Returns false if obj is already a member of the collection. or if the collection does not allow duplicates.

2. boolean addAll(Collection c) →

Adds all the elements of c to the invoking collection. Returns true if the operation succeeded otherwise returns false.

3. void clear() : →

Removes all elements from the invoking collection.

(5)

(V)

Page No.	
Date	

4. `boolean contains (Object obj) *` →
Returns true if `obj` is an element of the invoking collection otherwise returns false.

5. `boolean containsAll (Collection C) *` →
Returns true if the invoking collection contains all elements of `C` otherwise returns false.

6. `boolean equals (Object obj) *` →
Returns true if the invoking collection and `obj` are equal otherwise returns false.

7. `int hashCode () *` →
Returns the hash code of the invoking collection.

8. `boolean isEmpty () *` →
Returns true if the invoking collection is empty otherwise returns false.

9. `Iterator iterator () *` →
Returns an iterator for the invoking collection.

10. `boolean remove (Object obj) *` →
Returns true if `obj` is removed from invoking collection. It removes only one instance from the collection. Otherwise returns false.

(6)

Page No.

Date

11. `boolean removeAll (Collection c) *` → Removes all elements of `c` from the invoking collection. Returns `true` if the collection changed (i.e. elements are removed) otherwise returns `false`.

12. `boolean retainAll (Collection c) *` → Removes all elements from the invoking collection except those in `c`. Returns `true` if elements were removed otherwise `false`.

13. `int size () *` → Returns the number of elements held in the invoking collection.

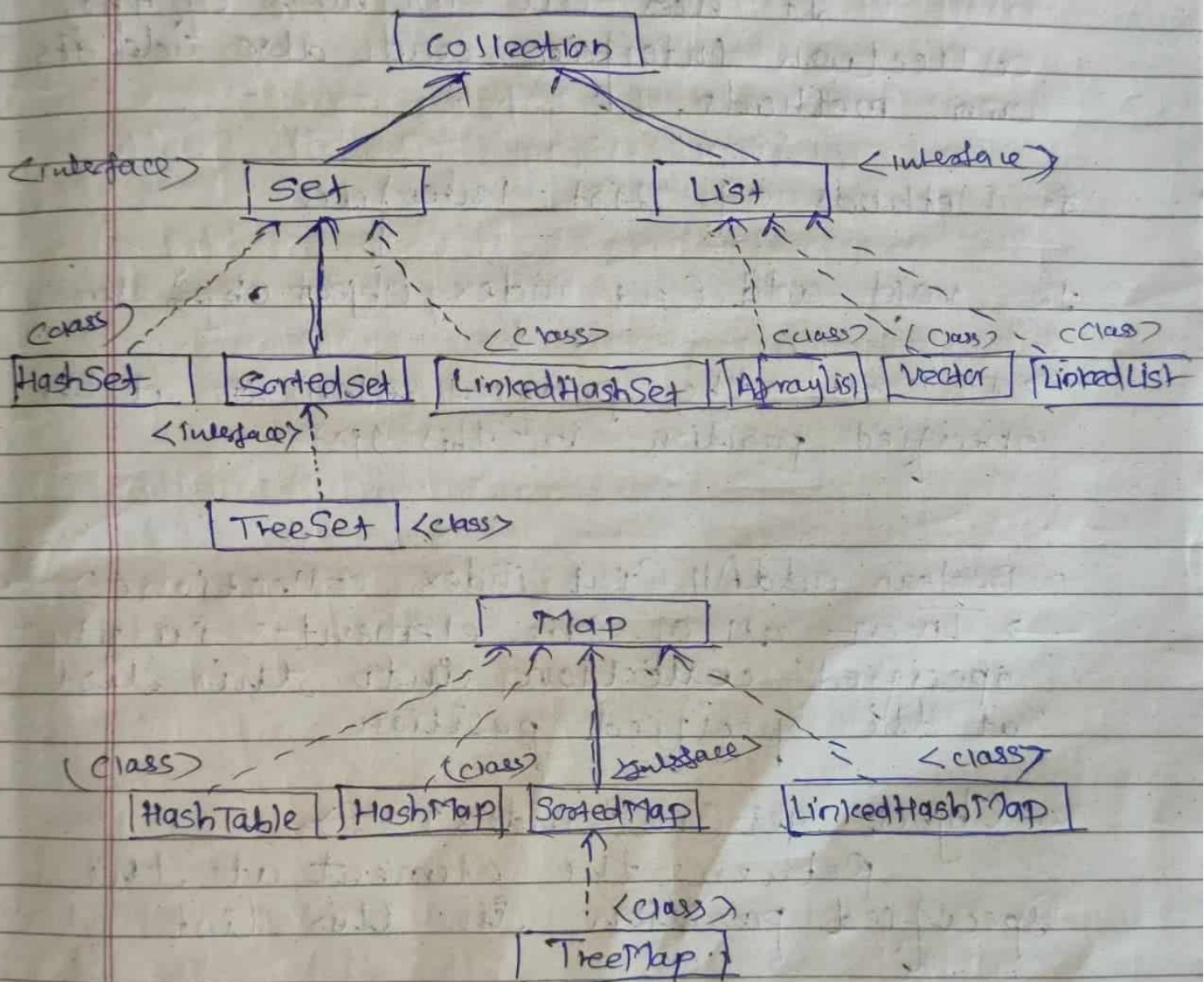
14. `Object[] toArray () *` → Returns an array that contains all the elements stored in the invoking collection. The array elements are copies of the collection element.

15. `Object[] toArray (Object array []) *` → Returns the array containing only those collection elements whose type matches that of array. If the size of array is less than that of number of matching elements, a new array of the necessary size is allocated and returned. If the size of array is greater than the number of matching elements the remaining are set to `null`.

(2)

Page No.			
Date			

* Classes Collection Classes * : →
 Above seen interfaces are implemented by several classes.
 Some of these classes are abstract i.e. they implement only a few methods



Collection Classes Hierarchy

(8)

* List * $\rightarrow$

A list is an ordered collection of objects (sometimes called a sequence).

List may contain duplicate elements.

The list interface extends the Collection interface.

It has all methods of the collection interface and also add its own methods.

* Methods of List Interface

1. void add (int index, Object obj) element

$\rightarrow$ It insert the specified element at the specified position in this list

2. Boolean addAll (int index, Collection c)

$\rightarrow$ Insert all of the elements in the specified collection into this list at the specified position

3. Object get (int index) $\rightarrow$

Returns the element at the specified position in this list

4. int indexOf (Object obj) * $\rightarrow$

Returns the index in this list of the first occurrence of the specified element or -1 if this list does not contain this element.

5. int lastIndexOf (Object o) * $\rightarrow$

$\rightarrow$ Returns the index of in this list

(9)

Page No.			
Date			

of the last element or -1. If this list does not contain the element

6. ListIterator listIterator() * →
Returns a list iterator of the elements in this list (in proper sequence).

7. Object remove(int index) * →
Removes the element at the specified position in the list.

8. Object set(int index, Object element) * →

Replaces the element at the specified position in this list with the specified element.

9. int size() * →
Returns the number of elements in this list.

10. list sublist(int fromIndex, int toIndex)
→ Returns a view of the portion of this list between the specified fromIndex inclusive, and toIndex exclusive.

* Features of List *

- 1) It stores a sequence of elements.
- 2) Elements can be inserted or accessed by their position in the list using an index (starting from 0).

(10)

Page No.	
Date	

8) A list may contain duplicate elements.

* ArrayList Class * →

ArrayList represents dynamic array that can grow or shrink as needed. An ArrayList is a variable-length array of object references.

ArrayList can be created with an initial size. As elements are added to an ArrayList its capacity grows automatically. When objects are removed from the ArrayList, it may be shrunk.

An ArrayList can hold objects of different types.

If we want to ArrayList to hold ~~same~~ type of objects, the objects of same type. Then we can specify the type while creating the ArrayList object.

* Constructors * →

1) ArrayList() :- It constructs empty list with an initial capacity of 10.

e.g.

```
ArrayList A1 = new ArrayList();
```

```
ArrayList<String> a1 = new ArrayList<String>();
```

(11)

Page No.			
Date			

- e. ArrayList(Collection c) : →
 It constructs an ArrayList containing the elements of the specified collection objects.

e.g.

```
ArrayList al = new ArrayList(c);
```

- f. ArrayList(int initialCapacity) * : →
 Construct an empty list with the specified initial capacity.

e.g.

```
ArrayList al = new ArrayList(50);
```

* Methods of ArrayList class are:-

1. add(Object e) → Appends the specified element to the end of the list.

2. add(int index, Object element) * : →
 Insert the specified element at the specified position in this list.

3. addAll(Collection c) * : →
 Appends all the elements in the specified collection to the end of this list. In the order that they are returned by the specified collection's iterator.

4. addAll(int index, Collection c) * : →
 Inserts all the elements of the specified collection into this list starting at the specified

(12)

Page No.	
Date	

position

5. clear() → Removes all of the elements from this list.

6. clone() * : —

Returns a shallow copy of this ArrayList ~~obj~~ instance.

7. contains(Object o) * : →

Returns true if this list contains the specified element.

8. ensureCapacity(int minCapacity) * : →

Increases the capacity of this ArrayList instance. If necessary, to

9. get(int index) : — Returns the element at the specified position in this list.

10. indexOf(Object o) — Returns the index of the ~~list~~ first occurrence of the specified element in this list or -1 if this list does not contain the element.

11. isEmpty() * : → Returns true if this list contains no elements.

12. iterator() : —

Returns an iterator over the elements in this list in proper sequence.

13. lastIndexOf(Object o) →

Returns the index of the last

(13)

Page No.			
Date			

occurrence of the specified elements in this list or -1 if this list does not contain the elements.

14. listIterator() : $\rightarrow$

Returns a list iterator over the elements in this list (in proper sequence)

15. listIterator(int index) * : $\rightarrow$ Returns a list iterator over the elements in this list (in proper sequence) starting at specified in the list

16. remove(int index) : $\rightarrow$ Removes the element at the specified position in the list.

17. removeAll(Collection c) * : $\rightarrow$

Removes from this list all of its elements that are contained in this specified collection.

18. removeRange(int fromIndex, int toIndex) * : $\rightarrow$

Removes from the list all of the elements whose index is between fromIndex inclusive and toIndex exclusive.

17. retainAll(Collection c) * : $\rightarrow$

Retains only the elements in this list that are contained in this specified collection.

18. set(int index, Object element) * : $\rightarrow$

Replaces the element at the specified position in this list with the specified elements.

(14)

Page No.	
Date	

19. `size()` * : $\rightarrow$
Returns the number of elements in this spec. list

20. `sort(Comparator c)` * : $\rightarrow$
sorts this list according to the order induced by the specified Comparator

21. `subList(int fromIndex, int toIndex int toIndex)` * : $\rightarrow$
Returns a new view of the portion of this list between `fromIndex` inclusive and `toIndex` exclusive

22. `toArray(T[] a)` * : $\rightarrow$ Returns an array containing all of the elements in this list in proper sequence (from first to last element)

23. `toArray()` * : $\rightarrow$ `—ll—`

24. ~~`trim()`~~ `trimToSize`

24. `trimToSize()` * : $\rightarrow$ Trims the capacity of the `ArrayList` instance to be the list's current size

* Program to demonstrate `ArrayList` operations
 $\rightarrow$

```
import java.util.*;
class ArrayListDemo
{
```

```
    public static void main(String[] args)
    {
```

(15)

Page No.			
Date			

```

ArrayList al = new ArrayList();
System.out.println("Initial size of ArrayList: "
    + al.size());
al.add("A");
al.add(20);
al.add(30);

System.out.println("Elements in ArrayList: " + al);

al.add(2, 36.8);
System.out.println("Elements in ArrayList: " + al);
System.out.println("Size after adding
    Elements: " + al.size());

al.set(3, 40);
System.out.println("ArrayList: " + al);

al.remove("A");

System.out.println("ArrayList: " + al);
al.removeAll();

```

* Sorting an ArrayList *: →

The object of an ArrayList can be sorted in two ways

1. Collections.sort() →

The collection framework provides a static method sort() to sort the objects of any Collection.

This method uses takes the collection object as an argument.

To sort the element in a descending order, use Collections.reverseOrder();

(16)

Page No.	
Date	

as the second argument. If the objects belong to user defined classes, then the sort method takes another argument in the form of a comparator object which specifies how the objects are to be compared.

e.g. `collections.sort(a1);`

1. `collections.sort(b, collections.reverseOrder())`

2. Using ArrayList class `sort()` * : —
It is used to sort the elements of an ArrayList.

* Advantages of ArrayList

1. The size of ArrayList can be changed during runtime.
2. It uses an array internally hence allows random access to any element.
3. An ArrayList allows insertion and deletion from any position.
4. An ArrayList can hold objects of different types.

* Disadvantages of ArrayList *
1. It allocates a particular amount of space according to the "capacity".

(17)

Page No.	
Date	

Once the no. number of elements exceed. The ArrayList has to copy the content array in to new memory space which takes time.

3. An ArrayList is not synchronized. This may cause problems when multiple threads are accessing the same ArrayList object.

Q. # LinkedList Class * →

→ It implements the List interface.
→ It implements all operations of List interface and permits all elements (including null). This class provides methods to get, remove, and insert an element at the beginning and end of the list.

* Constructors

1) LinkedList() → It constructs an empty list.

2) ~~LinkedList()~~

2) LinkedList(Collection c) * →
Constructs a list containing the elements of the specified collection.

* Methods of LinkedList Class * →

① add(Object o)

(18)

Page No.	
Date	

- ② add (int index, Object e)
- ③ addAll (Collection e)
- ④ addAll (int index, Collection c)
- ⑤ addFirst (Object e)
- ⑥ addLast (Object e)
- ⑦ clear()
- ⑧ clone() → Returns shadow copy of this list
- ⑨ contains (Object e)

→ 10) descendingIterator() → Returns the iterator over the elements in reverse sequential order.

→ 11. element() → Retrieves but does not remove the head (first element of the list)

12. get() → Returns the

→ 12. get (int index) → Returns the element at the specified position in this list.

- 13. getFirst()
- 14. getLast()
- 15. indexOf (Object e)
- 16. lastIndexOf (Object e)
- 17. listIterator (int index)
- 18. o

→ 18) offer (Object e) → Adds the specified element as the tail (last) element of this list.

(19)

Page No.			
Date			

- * OfferFirst(Object e) *! →
Adds the specified element at the front of this list.
- 20 OfferLast(Object e) *! →
Adds the specified element at the end of this list.
21. peek() *! → Retrieves but does not remove the head (first element) of this list.
22. peekFirst() *! → Retrieves but not removes the ~~list~~ first element of this list or returns null if this list is empty.
23. peekLast() *! → Retrieves but does not remove the last element of this list or returns null if this list is empty.
24. Poll() *! → Retrieves and removes the head (first element) of this list or returns null if this list is empty.
25. pollFirst() *! → Retrieves and removes the first elements of the list or returns null if this list is empty.
26. pollLast() *! →
Retrieves and removes the last element of this list or returns null if the list is empty.

(20)

Page No.

Date

27. `pop()` →
pops an element from the stack
representationed by the list

28. `push(Object e)` → —
Returns an element into the
stack represented by this list

29. `remove()` — Retrieves and removes
the head (first element) of this list

30. `remove(int index)`

31. `remove(Object e)` → ~~Retrieves and~~
~~removes~~ Removes the first occurrence
of the specified

~~30. `remove()` → Re~~

30. `remove(int index)` —

31. `remove(Object e)` —

Removes the first occurrence of
the specified object in this list
if it is present.

32. `removeFirst()` →

Removes and returns the first
element of this list.

33. `removeLast()` —

34. `RemoveFirstOccurrence(Object o)` →

Removes the first occurrence
of the specified element in this

(21)

Page No.	
Date	

list (when traversing list the list from head to tail).

35. `removeLastOccurance(Object e) *` →
Removes the last occurrence of the specified element in this list. Chooses traversing the list from head to tail.

36. `Set (int index object e) *` →

37. `size()`

38. `toArray()` — Returns an array containing all of the elements in this ~~list~~ list with ~~the~~ in proper sequence from first to last element.

39. `toArray(T[] a)` →
Returns an array containing all of the elements in this list in proper sequence (from first to last element).

* program to demonstrate linked list operation.

```

import java.util.*;
public class LinkedListDemo
{
    public static void main(String[] args)
    {
        LinkedList ll = new LinkedListDemo();
        ll.add("a");
        ll.add("b");
    }
}

```

(22)

Page No.	
Date	

```

ll.add(10);
System.out.println("Elements in Linked List are"
    + ll);
System.out.println("Size of linked list is: "
    + ll.size() + ll.size());
ll ll.addFirst(20);
ll.addLast("D");
ll
System.out.println("Elements of linked list"
    + " is: " + ll);
System.out.println("Size of linked"
    + " list is: " + ll.size());

```

* program to demonstrate

```

import java.util.*;
class demo
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        LinkedList ll = new LinkedList();
        LinkedList list = new LinkedList();
    }
}

```

```

System.out.print("Enter elements in linked list: ");
for (int i = 0; i < 10; i++)
{
    ll.add(sc.nextInt());
}

```

```

System.out.println("Entered elements in"
    + " linked list");

```

(23)

Page No.	
Date	

* program to demonstrate listIterator on LinkedList.



```

import java.util.*
class demo
{
    public static void main( String[] args)
    {
        LinkedList<String> ll = new LinkedList<String>()
        Scanner sc = new Scanner(System.in)
        System.out.println("Enter Elements in LinkedList");

        for(int i=0; i<10; i++)
        {
            ll.add(sc.next());
        }

        System.out.println("Elements in Linked Are:");

        // Forward Traversing
        <String> itr =
        ListIterator itr = new ll.listIterator()

        while (itr.hasNext())
        {
            String n = itr.next();
            System.out.print(n + " ");
        }

        // Backward Traversing
        ll.set(10);
    }
}

```

29.

Page No.	
Date	

```
while (ctr. hasPrevious()) {
```

```
    int n1 = ctr.previous();
    System.out.println(" " + n1);
}
```

```
}
}
```

* Advantages of LinkedList.

1. It is dynamic in size
2. Elements can be inserted and deleted from any position
3. LinkedList operations allow it to be used as a stack, queue or double-ended queue (deque)
4. A linked list can contain objects of different type.

* DisAdvantages :-

1. Element access is slower since it is a sequential access data structure.

* Difference between Arraylist and LinkedList

- 1.

(25)

Page No.			
Date			

Arraylist

Linked list

- | | |
|---|--|
| 1. Arraylist provides random access archite | 1. Linkedlist only provides sequential access. |
| 2. An Arraylist provides fast access to any elements | 2. Accessing the elements is slow as it traverse the list sequentially |
| 3. Insertion/Deletion in arraylist is difficult as because a lot of shifting is done internally to address the memory space. | 3. Insertion and deletion is very fast in a linkedlist. |
| 4. Arraylist stores the object references in consecutive locations. which | 4. Linked list stores each object sequen separately and make links them using references |
| 5. Arraylist are not synchronized | 5. Linkedlist are synchronized. |

(26)

Page No.			
Date			

⑧ * Set *! →

→ It is a collection that does not contain duplicate elements.

It models the mathematical type 'set'. It extends the collection interface but does not add any methods.

The set method contains only one method. The set interface contains only one method inherited from collection and adds restrictions that duplicate elements are prohibited.

⑨ * SortedSet Interface *! →

The sortedSet interface extends the set interface. It maintains its elements in ascending order, sorted according to the elements' natural ordering. In addition to those methods defined by Set.

* Methods of SortedSet Interface.

1. Comparator comparator() →

Returns the comparator associated with this sorted set or null if it uses its elements' natural ordering.

2. Object first() → Returns the first (lowest element) currently in this sorted set.

3. SortedSet headSet(Object toElement) → Returns a view of a portion of this sorted set whose elements are strictly less than toElement.

(22)

Page No.	
Date	

4. `Object last()` — Returns the last (highest) element currently in this sorted set.
5. `SortedSet subset (Object fromElement, Object toElement)` — Returns a view of the portion of this sorted set whose elements are range from `fromElement` inclusive, to `toElement` (exclusive).
6. `SortedSet tailSet (Object fromElement)` — Returns a view of the portion of this sorted set whose elements are greater than or equal to `fromElement`.

** `TreeSet` class →

~~This~~ This class implements the `SortedSet` interface.

The elements are internally stored in a search tree.

The internal data structure used by the `TreeSet` is a self-balancing search tree (red-black tree).

The `TreeSet` gives elements in sorted order (according to the natural order or by the of the elements or by the comparator provided by the at the set creation time).

It is useful when you need to extract elements from a collection in a sorted manner.

The main advantage of using a `TreeSet` is that basic operations (add, remove and contains) are performed in $O(\log n)$ time.

(28)

Page No.	
Date	

Program to demonstrate TreeSet

```
import java.util.*
public class demo
{
    public static void main(String[] args)
```

```

    {
        TreeSet ts = new TreeSet();
        ts.add("D");
        ts.add("C");
        ts.add("A");
        ts.add("R");
    }
```

```
    System.out.println("Elements of TreeSet: " + ts);
```

```
    System.out.println("The size of set is: " + ts.size());
```

```
    SortedSet S = ts.headSet("D");
```

```
    System.out.println("The elements < D are: " + S);
```

```
    S = ts.subSet("C", "S");
```

```
    System.out.println("The elements  $\geq C$  and  $< S$  are: " + S);
```

```
    S = ts.tailSet("D");
```

```
    System.out.println("Elements  $\geq D$  are: " + S);
```

```
    }
```

* Advantages *! →

- ① Allows only distinct elements.
- ② provides fast access to elements compared to linked list i.e $O(\log n)$ times.
- ③ stores the elements in the sorted order.

(29)

Page No.	
Date	

* Disadvantages:

- 1) It does not allow null elements, objects.
- 2) It has more overhead than arrays or HashSet.
- 3) Slower element access as compared to HashSet.
- 4) It is non-synchronized.

* HashSet Class * →

It creates a collection that uses the hash table for storage.

The advantages of HashSet is that it performs basic operations (add, remove, contains and size) in constant time, assuming that hash function distributes the elements properly among the buckets.

HashSet extends the AbstractSet and implements the Set interface.

* Constructors

1) HashSet() → constructs an empty HashSet.

2) HashSet(Collection c) → Construct a set containing the elements of the specified collection.

3) HashSet(int capacity) →

It initializes the capacity of HashSet to the specified capacity.

30

Page No.	
Date	

4. HashSet(int capacity, float loadFactor)
 — Construct a new, empty set with the specified initial capacity and the specified load factor.

The load factor must be between 0.0 and 1.0 and it determines how full the HashSet is before it is resized upwards.

Load factor is a measure of how full the HashSet is allowed to get before its capacity is increased.

When the number of entries exceeds the product of the load factor and the current capacity, the capacity is doubled.

e.g. If the initial capacity is 100 and the load factor is 0.75, then the HashSet will double in size to a capacity of 200 when the HashSet reaches 75 entries.

* Program to demonstrate some basic HashSet operations.

```
import java.util.*;

class Demo
{
    public static void main (String[] args)
    {
        HashSet hs = new HashSet ();
        hs.add ("B");
        hs.add ("A");
        hs.add ("F");
        System.out.println ("The content of
```

81

Page No.

Date

```

hash set : " + hs);
hs.add("E");
hs.remove("A");
system.out.println("The content of
hash set : " + hs);

```

```

TreeSet ts = new TreeSet(hs);
system.out.println("The tree set
contents are : " + ts);
}
}

```

* Advantages.

- 1) Allows only distinct elements.
- 2) Provides fast access to elements compared to TreeSet i.e $O(1)$ time.
- 3) Does not have a lot of overheads.
- 4) It is possible to optimize the memory usage by specifying the initial factor.
- 5) It allows null objects to be stored.

* Disadvantages.

- 1) The performance depends on the hash functions used.
- 2) It does not give elements in the order they were inserted i.e order does not preserved.
- 3) It does not give elements in the sorted order.
- 4) It is non-synchronized.

(32)

Page No.	
Date	

* Difference betⁿ TreeSet and HashSet

TreeSet	HashSet
① TreeSet stores elements in elements in red-black tree.	① HashSet stores it in a hashtable according to the hash function.
② Elements are ordered in a TreeSet	② Elements are not ordered in HashSet
③ Accessing elements in TreeSet is slower than HashSet i.e. $O(\log n)$	③ Accessing elements in HashSet is faster i.e. $O(1)$
④ To opt ⁿ No any tuning option to optimize space.	To optimize space HashSet tune the initial capacity and load factor.

* Map :-

Map

(33)

Page No.	
Date	

* Map —

Map interface —

A map represents an object that maps keys to values i.e. it stores key-value pairs.

The keys in map must be unique i.e. map cannot contain duplicate keys.

It does not extend the Collection interface.

The Java platform implements the contains the implementations of

- 1) HashMap.
- 2) TreeMap.
3. LinkedHashMap.

The Map interface provides methods to retrieve keys, values or the set of key-value mapping.

① void clear()

2) boolean containsKey(Object key) —
Returns true if this map contains a mapping for the specified key.

3. boolean containsValue(Object e) —
Returns true if this map contains a mapping for

Returns true if this Map maps one or more keys to the specified value.

34

Page No.	
Date	

4. Set `entrySet()` - Returns a set view of the mappings contained in this map.
5. `boolean equals(Object o)` - compares the specified object with this map its equality.
6. `Object get(Object key)` - Returns the value to which this map maps the specified key.
7. `int hashCode()` - Returns the hashcode value for this map.
8. `boolean isEmpty()` - Returns true if no key-value mapping is present in this Map.
9. `Set keySet()` - Returns a set view of the keys contained in this Map.
10. `Object put(Object key, Object value)` - Associates the specified value with the specified key in this Map.
11. `void putAll(Map c)` - copies all the mapping from the specified map to this map.
12. `Object remove(Object key)` - Removes the mapping for this key from this map, if it is present. (Optional operation)

(30)

Page No.	
Date	

10. `put size()` - It returns the number of key-value pairs mapping in this map.
11. `Collection values()` - Returns a collection view of the values contained in this map.

* TreeMap Class * -

This class implements a `Sorted` interface and stores the key-value pairs elements in a search (search tree). This ensures that retrieval is done very efficiently.

It uses a special type of search tree called as ~~red-black~~ tree. When the elements are retrieved they are retrieved in sorted order of keys.

* Constructors *

1. ~~TreeMap~~ `TreeMap()` -

`TreeMap()` - Constructs a new empty `TreeMap` with a ordering of keys as specified by the `comparator` object.

2. `TreeMap(Comparator c)` -

Construct a new empty `TreeMap` with a ordering of ~~keys~~ keys, as specified by the `comparator` object.

36

Page No.	
Date	

a. TreeMap(Map m) - Construct a treemap with all values from given map object ordered according to the natural ordering of the key.

b) TreeMap(SortedMap m) - Constructs a treemap with all values from a given SortedMap object ordering according to the ordering of its keys in SortedMap.

* Methods of the TreeMap class are -

1. Set entrySet() - Returns a set view of the mapping contained in this Map.

2. Object firstKey() - Returns the first (lowest) key currently in the sorted Map.

3. SortedMap & HeadMap(Object toKey) - Returns a partial view of portion of this map whose keys are strictly less than toKey.

4. Set keySet() - Returns a set of view of the keys contained in this Map.

5. Object lastKey() - Returns the last (highest) key currently in this sorted map.

(37)

Page No.	
Date	

7. SortedMap.subMap(Object fromkey, Object tokey) -
Returns a view of the portion of this map whose keys range from fromkey, inclusive to tokey, exclusive.

8. SortedMap.tailMap(Object fromkey) -
Returns a view of the portion of this map whose keys are greater than or equal to fromkey.

9. Collection values() -
Returns a collection view of the values contained in this treemap.

```

* import java.util.*;
public class TreeMapDemo
{
    public static void main(String[] args)
    {
        TreeMap tm = new TreeMap();
        tm.put
        tm.put("1", 35000);
        tm.put("2", 45000);
        tm.put("3", 25000);
        tm.put("A", 45000);
        tm.put("B", 65000);
        System.out.println("Elements in SortedMap
                             tm");
        collection
        Set keys = tm.keySet();
    }
}

```

(38)

Page No.	
Date	

```

system.out.println("keys are=" + keys);
Collection values = tm.values();
system.out.println("the values are=" + values);
}
SortedMap sm = tm.tailMap("m");
system.out.println("Entries having keys >= M are"
+ sm);
}
}

```

* Advantages -

1. Stores the key-value pairs in sorted order of keys.
2. Provides access to elements in $O(\log n)$ times.

* Disadvantages -

1. Slower access than HashMap
2. It is non-synchronized.
3. It does not allow null values.
4. Has more internal overhead.

* HashMap class! →

It internally uses the hashtable to implement the Map interface.

It stores key-value pairs in the hashtable.

(39)

Page No.	
Date	

keys are unique. It guarantees fast removal & retrieval of elements in $O(1)$ time.

It also permits null values and the null key.

The instance of HashMap has two parameters.

1. Initial Capacity - It is the number of buckets in the hash table and the initial capacity is a simple the capacity at the time the hash table is created.

2. load factor - Is a measure of how full the hash table is allowed to get before its capacity is automatically increased.

* Constructors:-

1. `HashMap()` -

Constructs an empty HashMap with the default initial capacity (16) and the default load factor (0.75).

2. `HashMap(int initialCapacity)` -

Constructs the empty HashMap with a specified initial capacity and the default load factor (0.75).

3. `HashMap(int initialCapacity, float loadFactor)` -

Constructs an empty HashMap with specified initial capacity and specified load factor.

(40)

Page No.	
Date	

4. HashMap (map m) -
Constructs a new HashMap with the same mapping as the specified Map.

* Program to demonstrate implementation of HashMap operations.

```

→ import java.util.*;
class Hashmapdemo
{
    public static void main(String [] args)
    {
        HashMap hm = new HashMap();

        hm.put("A", 35000);
        hm.put("B", 36000);
        hm.put("C", 45000);
        System.out.println("The elements in  
hashmap are =" + hm);

        Set keys = hm.keySet();
        System.out.println("Keys present in Hashmap  
are =" + keys);

        Collection values = hm.values();
        System.out.println("Values mapping to keys  
are =" + values);

        hm.remove("C");
        System.out.println(hm);
    }
}

```

(41)

Page No.

Date

* write a program to sort HashMap by keys and display the details before sorting and after sorting.

→ import java.util.*;
class Demo

{

public static void main(String[] args)

{

Map<Integer, String> hm = new HashMap<
Map<Integer, String> hm = new HashMap<
<Integer, String>()>

Scanner sc = new Scanner(System.in);
for (int i = 0; i < 5; i++)

{

System.out.println("Enter key
and value mapping")

int key = sc.nextInt();

String value = sc.next();

hm.put(key, value);

}

System.out.println("Elements of
HashMap before sorting")

System.out.println(hm)

TreeMap<Integer, String> tm = new TreeMap<
<Integer, String>(hm);

System.out.println("Elements After Sorting")

(42)

Page No.	
Date	

```

System.out.println(tm);
}
}
// Example
// System
// Set k = hm.keySet();
// Collection v = hm.values();
// System.out.println("Keys in HashMap are"
//                     + k);
// System.out.println("Value in HashMap"
//                     + value);
// }
// }

```

* Advantages of HashMap.

1. Provides fast access to elements compare to TreeMap. i.e. in $O(1)$ time.
2. Does not have a lot of overhead.
3. Possible to optimize the memory usage by specifying the initial capacity and load factor.
4. It allows null key and null values to be stored.

* Disadvantages -

- ① It does not give elements in the order they were inserted.
- ② It does not give the keys in sorted order.
- ③ It is not synchronized.

(33)

Page No.	
Date	

* Difference between TreeMap and HashMap

TreeMap HashMap

1. TreeMap stores key-value pairs in Red-Black tree. ① HashMap stores it in hashtable.

2) Keys are ordered in TreeMap. ② Keys are not ordered.

3) Slower element access than HashMap. 3) Fastest element access than TreeMap is in $O(1)$ time.
 i.e. in $O(\log n)$ time.

4. Can not tune to optimize storage or space usage. 4. Can optimize space usage by tuning initial capacity and load factor.

* Comparator Interface *

→ The Collections.sort() method sorts the objects of a collection by comparing the objects.

If the collection contains the objects of some other class of different type
e.g. (Student class having roll, name, age, percentage all having different data types.)

The sort method will not be able to sort these objects because it will

(44)

Page No.	
Date	

Not known how to compare the object.
 ∴ so we can compare the objects such objects by using `java.util.Comparator` interface that specify how the object should be compared.

The `Comparator` interface has one method `compare(Object o1, Object o2)`. This method takes two objects and parameters and returns a negative integer zero or a positive integer. If the first argument is less than equal to or greater than the second, it throws a `ClassCastException` if either of the arguments has an inappropriate type for the comparators.

e.g.

```
class Student
```

```
{
```

```
    String name;
```

```
    float per;
```

```
}
```

```
class CompareByPer implements Comparator
```

```
{
```

```
    <Student>  
    public static void main(String[] args)
```

```
{
```

```
    public int compare (Student o1, Student o2)
```

```
    { if (o1.per == o2.per)
```

```
        return 0;
```

```
    if (o1.per > o2.per)
```

```
        return 1;
```

```
    }
```

(45)

Page No.	
Date	

* Now if we want to sort according to percentage, we call the `collections.sort` method by passing an object of the `CompareByPerc` class.

```
ArrayList a = new ArrayList();
Collections.sort(a, new CompareByPerc());
Collections.sort(a, new CompareByPerc());
```

If we want to sort objects by names then

```
class CompareByName (Student a, Student b)
return (a.name.compareTo(b.name))
```

```
}
```

* Java program to demonstrate Comparator interface

→

```
import java.util.*;
class Student
{
    String name;
    float per;
    public String
    public Student (String name, float per)
    {
        this.name = name;
        this.per = per;
    }
}
```

(46)

Page No.	
Date	

```

public String toString()
{
    return "Name=" + name + "Per=" + per;
}
}

class SortByPer implements Comparator<String>
{
    public int compare (Student a, Student b)
    {
        if (a.per == b.per)
            return 0;
        if (a.per > b.per)
            return 1;
        return -1;
    }
}

class SortByName implements Comparator<String>
{
    public int compare (Student a, Student b)
    {
        return a.name.compareTo (b.name);
    }
}

```

```

class Demo
{
    public static void main (String[] args)
    {
        ArrayList<Student> a = new ArrayList<Student> ();
        a.add (new Student ("B", 87.5));
        a.add (new Student ("A", 75.4));
        a.add (new Student ("D", 90.2));
    }
}

```

(47)

Page No.

Date

```

system.out.println("Unsorted = " + a);
Collections.sort(a, new SortByPer());

```

```

system.out.println("Sorted By Percentages" + a);

```

```

Collections.sort(a, new SortByName());

```

```

system.out.println("Sorted By Name = " + a);

```

```

}
}

```

* Iterator

Java provides a standard mechanism to traverse any collection in an implementation dependent manner using an iterator.

The Iterator interface provides a methods using which we can traverse any collection.

The Iterator is also known as Universal Cursor of Java as it is appropriate for all the classes of collection framework.

This interface is implemented by all collections classes.

① boolean hasNext() → Returns true if this iterator has more elements.

② Object next() → Returns the next element in this list.

③ void remove(Object o) → Removes from the underlying collection the last element returned by the iterator.

(48)

Page No.			
Date			

```

import java.util.*;
class Demo
{
    public static void main (String[] args)
    {
        ArrayList al = new ArrayList();
        al.add(50);
        al.add(60);
        al.add(75);
        al.add(70.5);

        Iterator itr = al.iterator();
        System.out.println("ArrayList Elements are");
        while (itr.hasNext())
        {
            System.out.print(itr.next() + " ");
        }
    }
}

```

* ListIterator :-

It is a interface which extends Iterator interface

The LinkedList class implements this interface, using the `listIterator()` method.

Using the `ListIterator` we can traverse the LinkedList in both directions, forwards and backward.

(49)

Page No.	
Date	

It also allow the objects to be replaced by another object.

It can only be used on ArrayList and LinkedList classes but not on the set class type.

① void add(Object o) —

Insert the specified element in the list

② boolean hasNext() —

Returns true if this list iterator has more elements when traversing the list in the forward direction

③ int nextIndex() —

Returns the index of the element that would be returned by a subsequent call to next

④ Object previous() — Returns the previous elements in this list

⑤ int previousIndex() —

Returns the index of the element that would be returned by a subsequent call to previous

⑥ void set(Object o) —

Replaces the last element returned by next or previous with the specified elements.

(50)

Page No.	
Date	

Program to demonstrate listIterator methods.

```

import java.util.*;
class Demo
{
    public static void main(String[] args)
    {
        LinkedList ll = new LinkedList();
        ll.add("One");
        ll.add("Two");
        ll.add("Three");
        ll.add(50, "Four.");
        ListIterator itr = ll.iterator();
        while (itr.hasNext())
        {
            String n = itr.next();
            System.out.println(n);
            itr.set("Number " + n);
        }
        S.o.p("Traversing linkedList backward");
        while (itr.hasPrevious())
        {
            S.o.p(itr.previous());
        }
    }
}

```

(51)

Page No.	
Date	

O/P →

one

Two

Three

Four

Traversing Backward -

Number Four

Number Three

Number Two

Number One

* Advantages of Iterator

- ① We can traverse any collection in a standard manner without knowing the internal representation of elements.
- ② Iterator is fail-safe. It ensures that there are no concurrent modifications in the underlying collection which may happen in multi-threaded environment.
- ③ Using Iterator, we can read as well as remove the elements from the collection.

* Disadvantages

- ① Only a limited number of operations are supported by the Iterator.
- ② An Iterator can only be moved in forward direction while ListIterator can move in both directions.

(52)

Page No.	
Date	

* Enumeration -

The Enumeration is the another interface which can be used to enumerate or traverse through a set of elements.

It was developed before Iterator.

① boolean ~~hasNextElement~~

① boolean ~~hasMoreElements~~ () -

② Object ~~has~~ nextElement ()

* e.g

Enumerations e = Collections.enumeration

```
import java.util.*;
public class Demo
```

```
{
    public static void main(String[] args)
```

```
{
    ArrayList al = new ArrayList();
```

```
    al.add(50);
```

```
    al.add(10);
```

```
    al.add(20);
```

```
    Enumeration e = Collections.enumeration(al);
```

```
    System.out.println("Enumerates over list");
```

```
    while (e.hasMoreElements())
```

```
{
    s.o.p("value is " + e.nextElement());
}
```

(53)

Page No.	
Date	

O/P ⇒

p Enumeration over list

value is 50

value is 90

value is 20

Assignment - I

- Q.1 what is collections?
- Q.2 what are the collections Interfaces?
- Q.3 what are the collections classes?
- Q.4 write what are the components of Java Collection Framework?
- Q.5 what is Iterator.
- Q.6 what are the constructors of ArrayList class.
- Q.7 what is Set Interface.
- Q.8 write what are methods of
- Q.9 what is difference between List and Set
- Q.10 Difference between ArrayList and LinkedList
- Q.11 How we can sort an ArrayList?
- Q.12 write a program to implement LinkedList operations
- Q.13 write a program to implement TreeSet operations
- Q.14 what is HashMap class
write Note on HashMap class
- Q.15 write short Note on HashSet class
- Q.16 what is Comparator Interface

ch-2



* Multi-threading

Page No.	
Date	

* Introduction * →

Multi-threading is a specialized form of multitasking.

In multi-threading, there are several tasks that can be executed concurrently or in parallel within the same process or application.

Each task ~~can~~ which can run independently is called as Thread.

Java supports thread-based multitasking and provide high-level facilities for multithreaded programming.

* Threads →

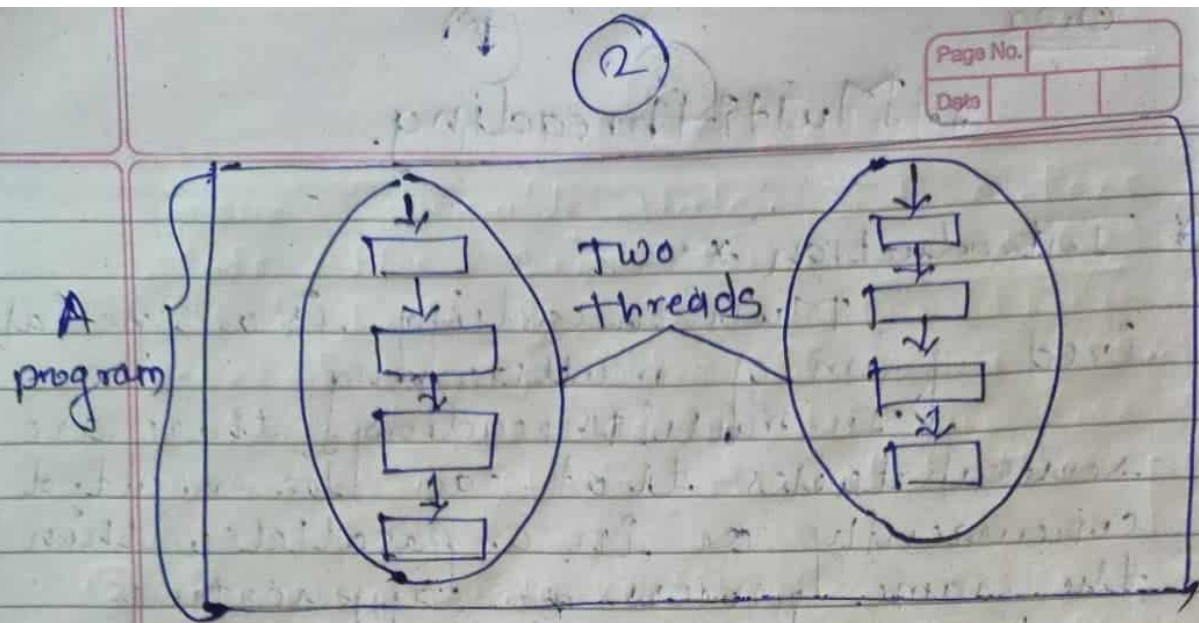
*** Def *** - Thread is a light-weight process that can be executed independently within an application. A thread defines a separate path of execution. A multithreaded program can run several threads concurrently.

In a single-processor computers the CPU executes each thread for a short time and then switches to the next thread.

In multi-processor computers, threads actually run in parallel.
e.g. A web browser.

A thread is similar to the sequential program. It has a beginning, a sequence and the end.

However a thread itself is not a program, a thread cannot run on ~~itself~~ its own. Rather it runs within a ~~method~~ program.



Two threads running concurrently in a single program.

* Advantages of multithreading

1. The CPU Idle time is kept to a minimum.
2. Efficient use of computing and I/O resources.
3. Threads can be started and stopped independently.

Simultaneous execution of several tasks thereby ~~improving~~ improving the performance of the application.

* Disadvantages

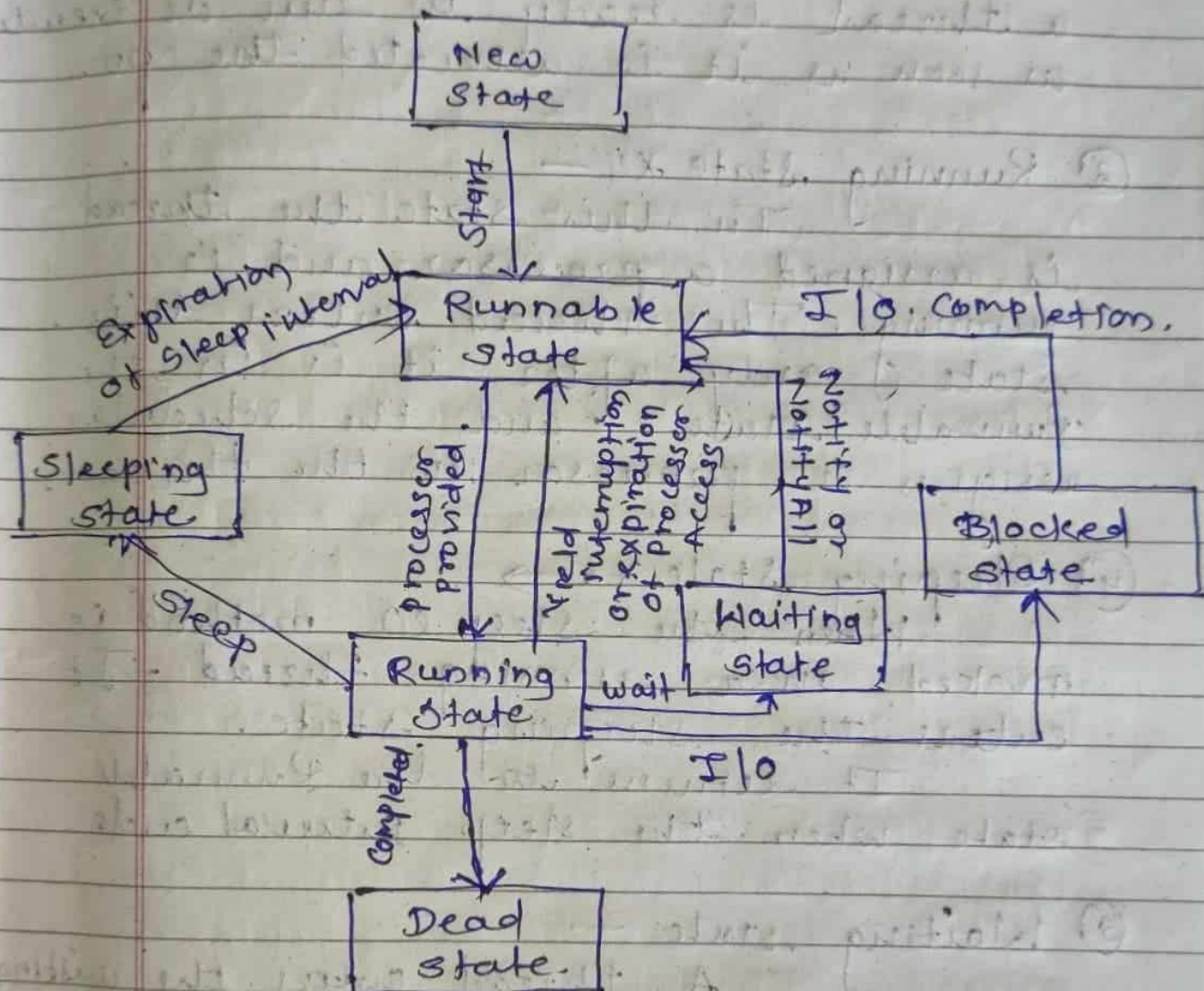
- ① Multithreaded applications are difficult to code.
- ② Difficulty in debugging.
- ③ Difficult to manage the concurrency and requires precise control and context switching.

(3)

Page No.	
Date	

* Thread Lifecycle *

The lifecycle of threads consists of several states which a thread can be in, each.



1. New state *

When a thread object is created, it is in the new state. It is in this state before the start method is invoked. At this state point, the thread is considered not alive.

2. Runnable or Ready state

A thread starts its lifecycle

(A)

Page No.	
Date	

from Runnable state. It enters this state the first time `start()` is called but it can enter this state several times later. In this state a thread is ready to run or execute as soon as it is allocated the CPU.

② Running state :-

In this state the thread is assigned a processor and is running. The thread enters this state only after it is in the Runnable state and the scheduler assigns a processor to the thread.

④ Sleeping state :-

When the `sleep()` method is invoked on a running thread. It enters the sleeping state.

It returns to the Runnable state when the sleep interval ends.

⑤ Waiting state :-

A thread enters the wait state when the `wait()` method is invoked on the thread. When some other thread issues a `notify()` or `notifyAll()` it returns to the Runnable state.

⑥ Blocked state :-

A state can enter this state because it is waiting for resources that are held by another threads typically I/O resources.

(5)

Page No.	
Date	

① Dead state :-

This is the last state of a thread type. When the thread has completed execution i.e. its `run()` method ends the thread is terminated.

* Creating and Running Thread

* $\rightarrow$

There are two ways in which we can create a thread in Java

1. Implementing the Runnable interface
2. Extending the Thread class.

① Implementing Runnable Interface :-

The `java.lang.Runnable` interface can be used to create a thread.

It has only one method named `run()`. The `run()` method defines the thread actions.

```
public interface Runnable
{
    void run();
}
```

To create a thread using a Runnable interface, the following steps should be performed:

1. Define a class, which implements

(6)

Page No.	
Date	

Runnable Interface

2. In this class define the `run()` method which specifies the thread actions.
3. Create an object of this class.
4. Create an object of the `Thread` class and pass the `Runnable` object to the constructor.
5. Invoke the `start()` method on the `Thread` object to execute the thread.
This method spawns a new thread i.e. the new thread will begin execution as a child thread of the current thread.
The `start()` method internally calls the `run()` method.

Code Outline

```

class myRunnableDemo implements Runnable
{
    public void run()
    {
        s.o.p.c() "message";
    }
}

class Demo {
    class public static void main (String[] args)
    {
        myRunnableDemo r = new myRunnableDemo();
        Thread t = new Thread(r);
        t.start();
    }
}

```

(7)

Page No.	
Date	

Program to create Thread by implementing Runnable interface

→ class myNumber implements Runnable

```
{
    public void run ()
```

```
{
```

```
    for (int i = 0; i < 10; i++)
```

```
{
```

```
        System.out.println("Number: " + i);
```

```
        System.out.println("Number: " + i);
```

```
    }
```

```
}
```

Class Demo

```
{
```

```
    public static void main (String[] args)
```

```
{
```

```
        myNumber mn = new myNumber();
```

```
        Thread t = new Thread (mn);
```

```
        System.out.println ("t");
```

```
        t.start ();
```

```
    }
```

```
}
```

3)

Output → Number: 1

Number: 2

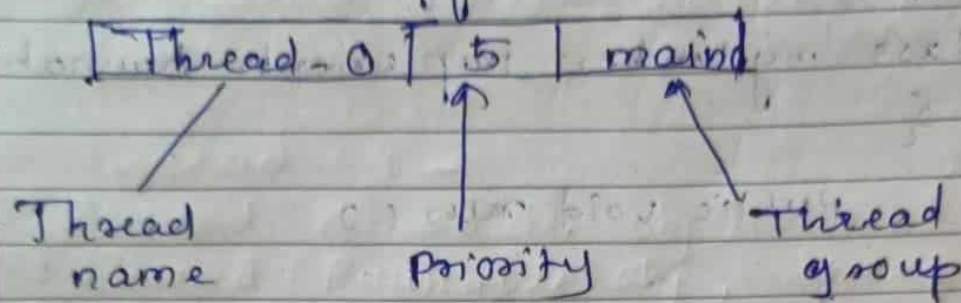
Number: 3

Number: 10

(8)

Page No.	
Date	

The first line of output displays the thread information.



II) Extending Thread Class :-

The second method to create thread is by extending Thread class. The java.util.Thread class implements the Runnable interface and contains many methods to perform operations on a thread.

However, the run() method in the Thread class is an empty method i.e. it does nothing. When we extend the Thread class we should override the run method to specify the thread actions.

* Example Code Outline

```

→ class myThread extends Thread
{
    public void run(); // override
    // Define thread action.
}

```

(9)

Page No.	
Date	

```
myThread m = new myThread();
    m.start();
}
```

* program to extend Thread class.

```
import java.util.*;
class myThread extends Thread
{
```

```
    public void run()
    {
```

```
        for(int i = 0; i < 5; i++)
```

```
        {
            System.out.println("Hello!!");
        }
    }
}
```

Class Demo

```
public static void main(String[] args)
{
```

```
    Thread t = new Thread();
```

```
    t.start();
}
```

O/P → Hello: 0

Hello: 1

Hello: 2

Hello: 3

Hello: 4

The Constructors.

The Thread class contains many constructors

- 1) Thread () -
- 2) Thread (Runnable obj) -
- 3) Thread (Runnable obj, String name) -
- 4) Thread (String name) -
- 5) Thread (ThreadGroup grp, Runnable obj, String s) -
- 6) Thread (ThreadGroup grp, String name) -
- 7) Thread (ThreadGroup grp, Runnable obj, String name, long nsec) -
- 8) Thread (ThreadGroup grp, Runnable obj) -

* Thread class Methods →

- 1) static int activeCount () -

Returns the no. of active threads in the current threads, thread groups.

- 2) static Thread CurrentThread () -

Returns a reference to the current executing thread object.

- 3) String getName () - Returns this thread's name.

- 4) int getPriority () - Returns the thread's priority.

- 5) ThreadGroup getThreadGroup () - Returns the threadgroup to which this thread belongs.

Page No.	
Date	

- 6) void interrupt() - ~~Returns~~ Interrupts this thread.
7. boolean isAlive() - Tests if this thread is alive.
8. ~~boolean isInter interrupted()~~
8. boolean isInterrupted() - Tests whether the thread has been interrupted.
9. void join() - Waits for this thread to end.
10. void setName(String name) - Changes the name of this thread.
11. void setPriority(int newpriority) - Changes the thread priority of this thread.
12. void sleep(long msec) - Causes the current executing thread to sleep for given time span in milliseconds.
13. void start() - Causes this thread to begin execution.
14. String toString() - Returns a string representation of this thread including thread's name, priority and thread group.

(10)

Page No.	
Date	

15. Static void yield() —

Causes the currently executing thread object to temporarily pause and allow other threads to execute.

* program to demonstrate Thread class methods -

```
class myThread extends Thread
{
```

```
    void run()
    {
```

```
        System.out.println("Hello, Thread is executing");
    }
```

```
}
```

```
class demo ThreadDemo
```

```
{
    public static void main (String[] args)
    {
```

```
        myThread t = new myThread();
        t.start();
```

```
        System.out.println(t);
```

```
        System.out.println("Thread Name before change is = " + t.getName());
```

```
        System.out.println("Thread priority = " + t.getPriority());
```

```
    }
}
```

(11)

Page No.	
Date	

~~System.out.out.pr~~

```
t.setName("NewName");
system.out.println("Thread Name After Change"
    + t.getName());
```

~~system.out~~

```
t.setPriority(Thread.MAX_PRIORITY);
system.out.println("New Thread priority is="
    + t.getPriority());
}
```

* Thread Priorities *

Every thread has a priority. Thread priorities play an important role in scheduling threads. Higher priority thread gets more execution time.

A priority is an integer from 1 to 10 inclusive, where 10 is the highest priority referred to as the maximum priority and 1 is the lowest priority referred to as a minimum priority. The normal priority is 5 which is the default priority.

To find the priority of the thread use the `getPriority()` method. To set the thread priority use the `setPriority()` method by using integer from 1 to 10.

(12)

Page No.	
Date	

1. Thread.MIN_PRIORITY : Minimum priority
i.e. 1

2. Thread.MAX_PRIORITY : Maximum priority
i.e. 10

3. Thread.NORM_PRIORITY : Normal or Default
priority i.e. 5.

* The main Thread *

Every java program runs as a Thread. When a program starts running a thread called 'main' starts running. When this thread stops running, the program terminates. The main thread spawns other threads - called child threads.

```
public class currentThreadDemo
{
    public static void main(String[] args)
    {
```

```
        Thread t;
```

```
        Thread t = new Thread();
```

```
        Thread t = Thread.currentThread();
```

```
        System.out.println("Current Thread Name = "
```

```
            + t.getName());
```

```
        System.out.println("priority of Thread = "
```

```
            + t.getPriority());
```

```
        if (t.isAlive())
```

```
        {
            System.out.println("Main Thread  
is alive");
        }
```

}

(13)

Page No.	
Date	

else

System.out.println("Thread is not alive");

}

}

O/P →

Current Thread Name :- main

priority = 5

~~main Thread~~

main thread is alive

* sleep() method * →

The sleep() method causes a thread to sleep for a specified number or amount of milliseconds. We can pause a running thread using the sleep() method. However the sleep() method throws an InterruptedException when it is interrupted by another thread. Hence we should handle the exception when the sleep method is used.

* Program to demonstrate threads with sleep().

• Class mythread extends Thread.

{

String msg;

mythread(String msg)

{

this.msg = msg;

}

public void run()

(14)

Page No.	
Date	

```

{
    try
    {
        for (int i = 0; i < 5; i++)
        {
            System.out.println("msg " + i);
            Thread.sleep(1000);
        }
    }
    catch (Exception e)
    {
    }
}

```

```

public class ThreadDemoSleep
{
    public static void main (String[] args)
    {
        mythread t = new mythread("Hello");
        t.start();
    }
}

```

- * Running Multiple Threads *:-
- It is possible to create multiple threads within a program. Each thread performs an independent task and runs independently.
- A thread scheduler manages the execution of threads and assigns the CPU to each thread. The order in which the thread executes will depend upon

(15)

Page No.	
Date	

```

//msg
//due cpu and the thread scheduler
import java.util.*;
class MultipleThread extends Thread
{
    String msg;
    int n;
    MultipleThread(String msg, int n)
    {
        this.msg = msg;
        this.n = n;
    }
    public void run()
    {
        try
        {
            for (int i=0; i<n; i++)
            {
                System.out.println(msg + " : " + i + " time");
                Thread.sleep(2000);
            }
        } catch (InterruptedException e)
        {
        }
    }
}

//Class Demo
{
    public static void main(String[] args)
    {

```

(16)

Page No.	
Date	

```

multipletHread t1 = new multipletHread("
COVID19", 10);
t1.start();

```

```

multipletHread t2 = new multipletHread(
"LOCKDOWN2020", 20);
t2.start();

```

```

multipletHread t3 = new multipletHread
("VACCINATED2021", 30);
t3.start();

```

```

}
}

```

will print

O/P $\Rightarrow$ LOCKDOWN 10 times
 LOCKDOWN2021 20 times
 VACCINATED 2021 30 times
 Randomly.

* Synchronization *

In multithreaded applications two or more threads may access shared resources like objects, memory database or data structures.

Each thread may access the resource simultaneously and also modify the state of the shared resources.

In this case, it may leave the resource in an inconsistent state.

(17)

Page No.	
Date	

such a situation is called as race condition. In such cases, it is important to ensure that the resource is used by only one thread at a time. The mechanism to achieve that is called synchronization.

To avoid the Race Condition we can use synchronized block or methods must be declare as synchronized.

e.g.

```
class producer extends Thread
{
```

```
    public void run()
    {
```

```
        synchronized (cal)
        {
```

```
            for (int i = 0; i < cal.size(); i++)
```

```
            {
```

```
                cal.add(obj);
```

```
            }
```

```
        }
```

```
    }
}
```

→ syntax for synchronized method.

(18)

Page No.

Date

→ `public synchronized method_name (arguments)`

```
{
    // code
}
```

→ `synchronized (object expression)`

```
{
    // code.
}
```

e.g.

```
public synchronized void producer(al)
{
    // code
}
```

```
public synchronized (obj)
{
    // code
}
```

* **Enter** Write a program for synchronized i.e. producer-consumer problem.

→ `class producer extends Thread`

```
{
    ArrayList al = new ArrayList<Integer> al = null;
    final int capacity = 5;
}
```

(19)

Page No.	
Date	

```

private int i = 0;
public producer (ArrayList<Integer> a1)
{
    this.a1 = a1;
}

public void run ()
{
    while (true)
    {
        try
        {
            produce (i++);
        } catch (Exception e) {}
    }
}

```

```

public void produce (int i) throws
    Exception

```

```

{
    synchronized (a1)
    {
        while (a1.size() == capacity)
        {
            System.out.println("Shared List is full
            ... waiting for consumer to
            consume");
            a1.wait();
        }
    }
}

```

```

synchronized (a1)
{
    System.out.println("producer produce
    the element = " + i);
}

```

(20)

Page No.	
Date	

```

    al.add(c);
    Thread.sleep(500);
    al.notify();
}
}

```

```

class consumer extends Thread
{

```

```

    ArrayList<Integer> al = null;

```

```

    public consumer(ArrayList<Integer> al)
    {
        this.al = al;
    }

```

```

    public void run()
    {

```

```

        while (true)
        {

```

```

            try
            {

```

```

                consumer();
            }

```

```

            catch (Exception e) {}
        }
    }
}

```

```

}
}

```

```

    public void consumer () throws Exception
    {

```

```

        synchronized (al)
        {

```

```

            while (al.isEmpty())
            {

```

```

                System.out.println("Shared list is empty
                ...wait for producer to produce
            }
        }
    }
}

```

```

        al.wait();
    }
}

```

```

}
}

```

(21)

Page No.	
Date	

```

synchronized (al);
{
    Thread.sleep(500);
    System.out.println("Consumer the
    element = " + al.remove());
    al.notify();
}
}

```

```

public class prod_cons
{
    public static void main(String[] args)
    {
        ArrayList<Integer> al = new ArrayList<>();
        Producer p = new Producer(al);
        Consumer c = new Consumer(al);

        p.start();
        c.start();
    }
}

```

(22)

Page No.	
Date	

* Interthread Communication *! →

When multiple threads are running in an application, it becomes necessary for the threads to communicate with each other. When threads within a single program talk to one another, it is called inter-thread communication.

The threads can communicate by either sharing data or indicating to other threads when a specific activity has completed.

1. Using `join()` —

The simplest way for interthread communication is to use the `join()` method. This method makes the thread wait until the completion of another thread before continuing.

2. `join()` & Using `wait()`, `notify()`, `notifyAll()` —

These three methods of the `Object` class must be used within a synchronized block. The `wait()` method makes the thread wait, the `notify()` and `notifyAll()` methods inform the waiting threads of completion of the activity.

* `Join()` method *! —

`Join` method causes a thread to wait for another thread to finish. Suppose the `join()` method is called for a thread `t1`. It causes the current thread `t1` on which the `join()`

(23)

Page No.	
Date	

is called) terminate. The join method can also be called for multiple threads. It can also be called with a time out value in milliseconds, for eg. -
 void join(long timeout);

e.g.

```
class myThread extends Thread
{
    int sum = 0, pos;
    myThread (int p, int [] a)
    {
        arr = a;
        pos = p;
    }
    public void run()
    {
        for (int i = pos; i < pos + 100; pos++)
        {
            sum = sum + arr[i];
        }
    }
    int getSum()
    {
        return sum;
    }
}

class joinDemo
{
    public static void main (String [] args)
    {
```

29

Page No.	
Date	

```

int sum = 0;
int[] myArray = new int[1000];
for (int i = 0; i < 1000; i++)
    myArray[i] = i + 1;

MyThread[] t = new MyThread[10];
for (int i = 0; i < 10; i++)
{
    t[i] = new MyThread(i * 100, myArray);
    t[i].start();
    t[i].join();
}
for (int i = 0; i < 10; i++)
{
    sum += sum + t[i].getSum();
}
System.out.println("Sum: " + sum);
}

```

Ch. 3.

* Database Programming

Page No.	
Date	

A database is an organized collection of data.

A database management system provides mechanism for storing, organizing and modifying data from a database.

A java program may need to access and manipulate the data stored in a database. Hence the program must communicate with the database.

This is possible using the JDBC API. The JDBC API is a java API that can access any kind of tabular database especially data stored in a relational database.

* The Design of JDBC *! →

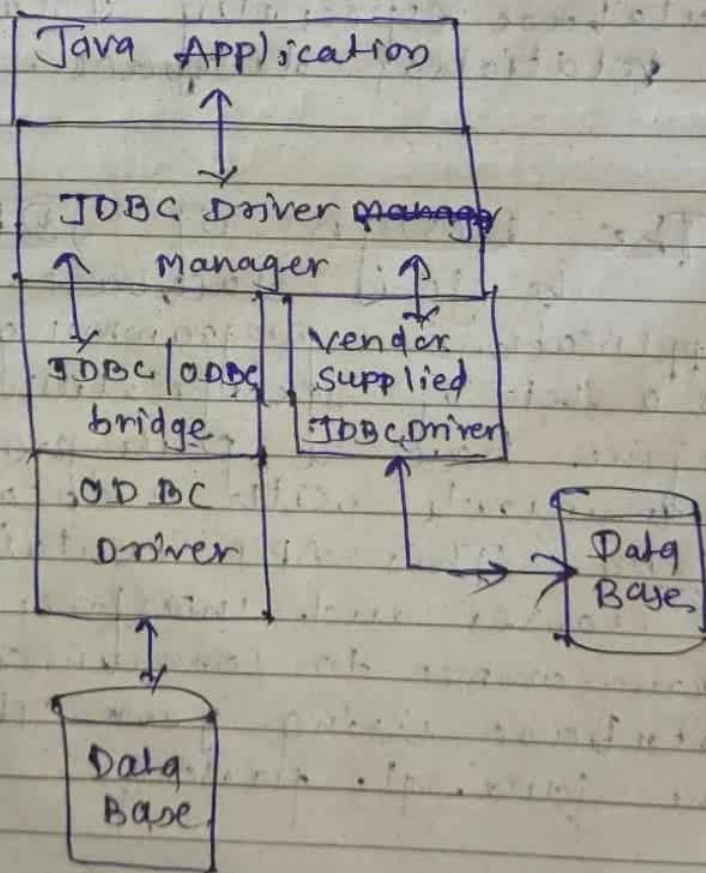
The java database connectivity Applications programming Interface is a set of specifications that define how a java program can communicate with a database.

Java. This API contains a set of classes and interfaces to enable programmer to communicate with a database using java. These are in the java.sql package.

It defines how an application opens a connection, communicates with the data base, executes sql statements and retrieves the result.

What does the JDBC API does
 → The JDBC API makes it possible to do three things

1. Establish a connection with a data source.
2. Send queries and update statements to the data source.
3. Process the results.



The design of JDBC.

Page No.			
Date			

* JDBC Drivers * -

A JDBC driver allows the Java application to communicate with the database.

It also ensures that the requests made by the application are presented to the database in a language that is understood by the database.

It does following task.

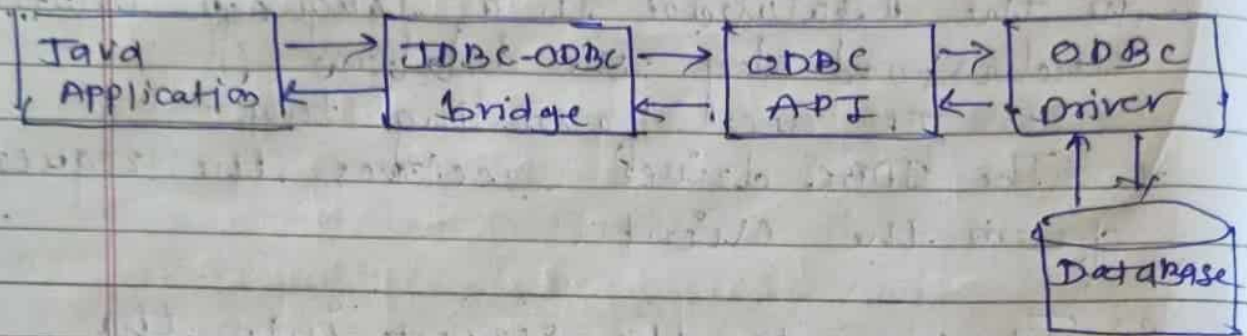
1. The JDBC driver receives the request from the client.
2. It converts the request into the format understandable by the database.
3. It then submit the request to the database.
4. The JDBC driver receives the response from the database.
5. It translates the response back to the Java data format.
6. It submit the response to the client applications.

* Types of Drivers *

JDBC drivers can be classified into four categories.

1. Type - 1: JDBC-ODBC Bridge Driver :-
 → The JDBC-ODBC bridge driver connects Java programs to ODBC (Open database Connectivity) data sources.
 This driver typically requires

an ODBC driver on the client computer and requires an ODBC driver on the client computer and requires configuration of ODBC data sources.



JDBC-ODBC Bridge Drivers

* Advantages —

It can be used to access any database for which the ODBC driver is installed.

* Disadvantages * —

1) Since the bridge driver is not written fully in java, type-safe drivers are not portable.

2. The JDBC call goes through the bridge to the ODBC driver, then to the database and this applies even in reverse process. Hence they are the slowest of all driver types.

3. The ODBC driver needs to be installed on the client machine.

4. The ODBC driver need to be installed.

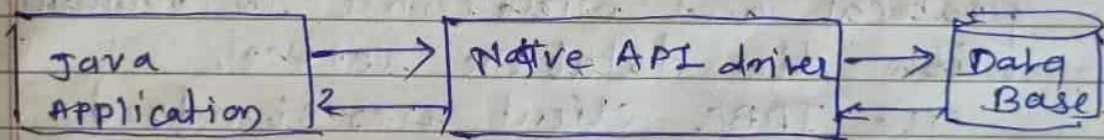
5. Not suitable for the web.

2) Most of the data sources supported ODBC access earlier. However, it is no longer in use.

2) TYPE-2 Native-API driver : →

The Native-API driver is a partly-Java driver that enables Java programs to communicate with the database using database specific APIs, which are normally written in C or C++ known as native code.

This style of driver requires that some operating system-specific native code should be loaded on each client machine.



Native-API Driver

Advantages -

1) This driver offers better performance than the JDBC-ODBC bridge since no translation is required.

2) It uses Native Native API which is database specific, hence communication is faster.

Disadvantages :

1) Native API must be installed on the client machine.

2) Can not be used for the Internet.

Since the native API should be installed in the client machine

3. Like Type-1 drivers, it is not written in Java language which forms a portability issue

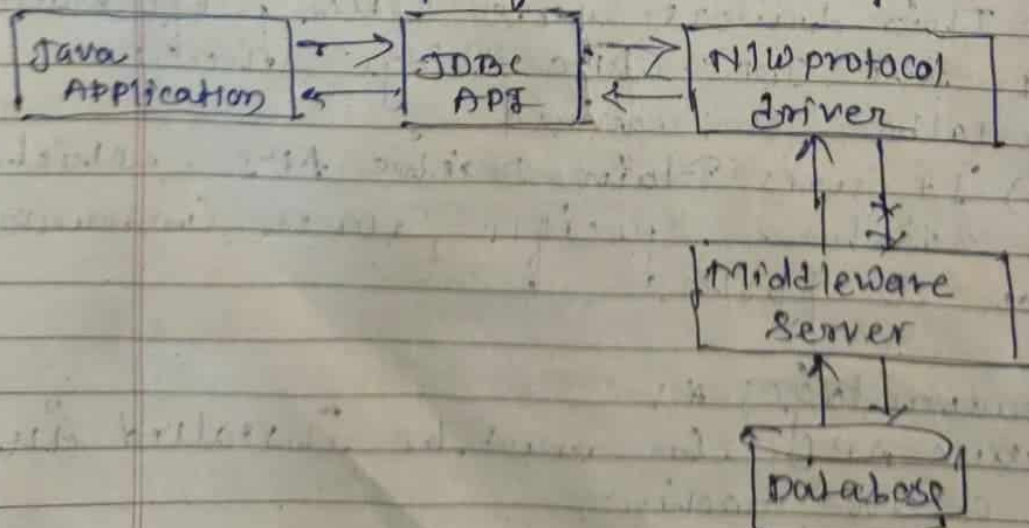
4. If the Database changes, the client library should also be changed.

5. Usually not thread safe.

3] Type-3 : Network protocol Driver:-

The driver uses a networking protocol and middleware to communicate with the database server.

The JDBC Client uses standard network socket to communicate with a middleware applications server which then translate it into the vendor-specific database protocol.



Network Protocol Driver

Page No.	
Date	

* Advantages.

1. Since the communication between client and the middleware server is database independent there is no need for the vendor library on the client machine.
2. This driver is fully written in java and hence portable.
3. The client driver need not be changed for a new database.
4. It is suitable for the web.
5. The client JDBC driver is very small and fast to load.
6. They are the most efficient amongst all drivers.

* Disadvantages —

- 1) It requires a server application to be installed and maintained.
- 2) Accessing the data may take longer since it is a three-tier approach.

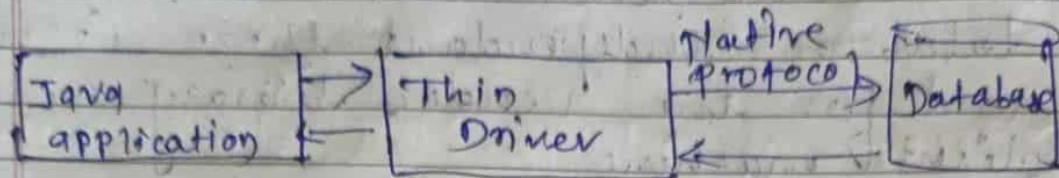
4. Type-4 Thin Driver *! →

It is also called Native protocol driver. This driver is a 100% pure java driver that converts JDBC calls directly into the vendor-specific database protocols.

This allows a direct call from the client machine to database server.

Type-4 drivers are entirely written in java and communicate directly

* with the database through socket connection



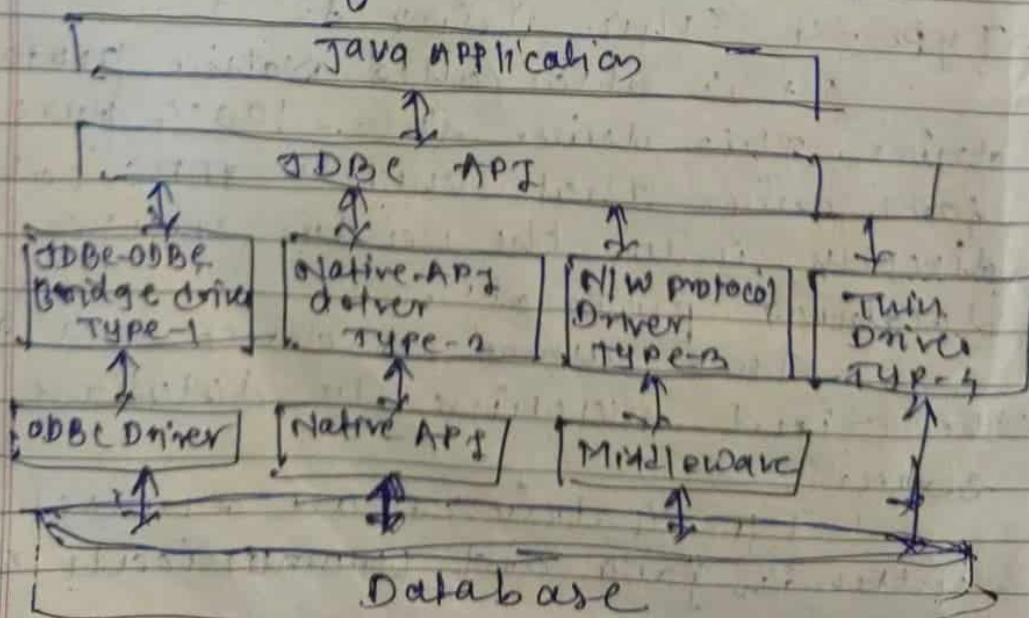
Thin Driver

* Advantages -

1. They are written in java and hence portable and platform independent
2. The number of translations are very less. i.e. type 4 JDBC drivers don't have to translate database request in ODBC or pass the request on to another server.
3. Performance is very good.

* Disadvantages -

- 1) At the client side a separate driver is needed for each database



* Standard JDBC Drivers

DBMS	JDBC Driver Name	Database URL Format
1. DB2	com.ibm.db2.jcc.DB2Driver	"jdbc:db2: //<HOST>[: <PORT>]<DB>"
2. PostgreSQL	org.postgresql.Driver	"jdbc:postgresql: //<Host>[:<port>] <DB>"
3. MySQL	com.mysql.jdbc.Driver	"jdbc:mysql://<Host> /<DB>"
4. ORACLE	oracle.jdbc.driver. OracleDriver	"jdbc:oracle: thin:@<HOST> :<PORT>:<DB>"
5. Sybase	com.sybase.jdbc. SybaseDriver	"jdbc:sybase: Tds:<HOST>:<PORT> > /<DB>"
6. MS-Access	net.ucanaccess. JdbcUcanaccessDriver	"jdbc:ucanaccess: :<DB>"
7. Microsoft SQL server	com.microsoft.sqlserver. JdbcSQLServerDriver	"jdbc:sqlserver ://<Host>"

* JDBC classes and Interfaces.

1. The java.sql package contains several classes and interfaces which are used to communicate with a database.

Name of Interfaces	Description
1. Connection	Represent a connection session with a database.
2. DatabaseMetaData	Information about the database.
3. Driver	Interface that every driver class must implement.
4. PreparedStatement	Represents for precompiled SQL statements.
5. ResultSet	Table of data generated by executing a database query.
6. ResultSetMetaData	Information about columns in a ResultSet.
7. Statement	For executing a static SQL statements and returning the results it produces.

* Classes — Description

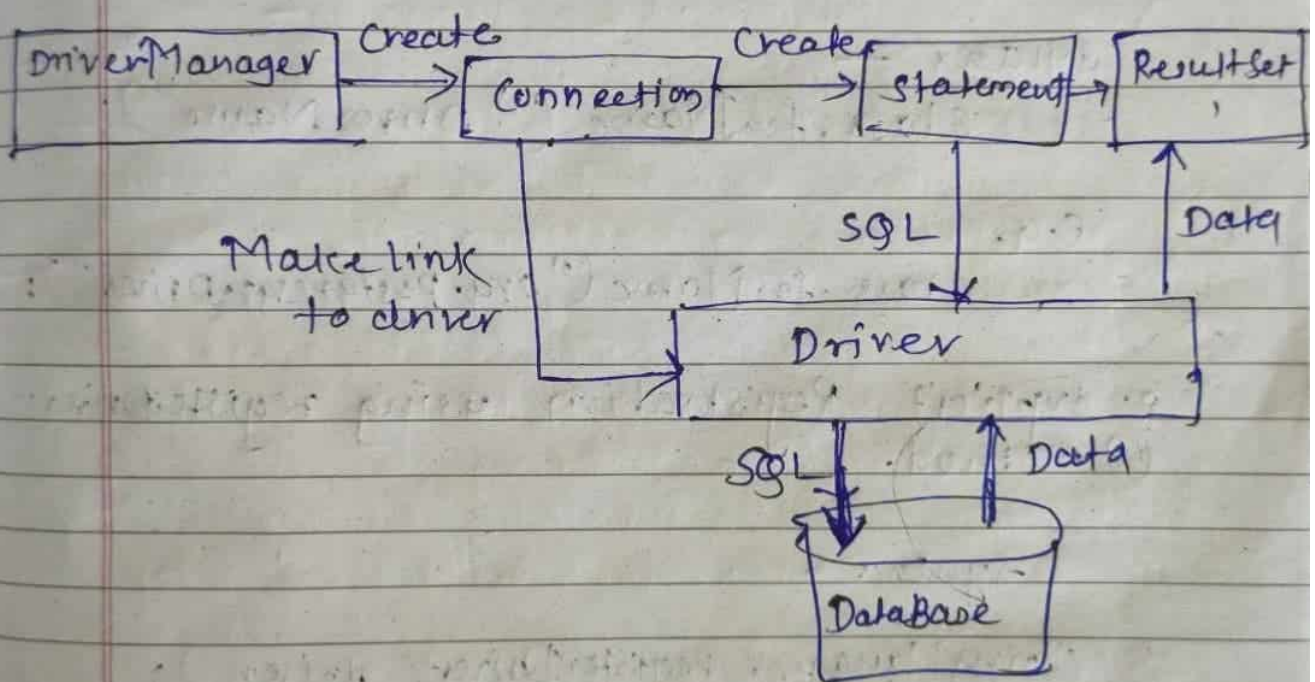
1. DriverManager	The basic service for managing a set of JDBC drivers.
2. Date	Represent an SQL date values.
3. Time	Represent an SQL time values.

8. Executing SQL Statements and Queries -

*:-

An JDBC program follows some basic steps to communicate with a database

1. Load the driver
2. Establish the connection
3. Create a statement object
4. execute a query
5. process the query, the result
6. close the connection.



JDBC Steps

Step-1 :- Load the Driver -

* Using `Class.forName()` method -

To communicate with any database we have to use a driver. Before we can use a driver we must be registered with a JDBC DriverManager.

Page No.	
Date	

The DriverManager class maintain the list of Driver classes that have been registered.

1. Using Class.forName() Method —
 → It is a static method of the Class class. This method loads the specified driver class ~~with the~~. When the getConnection() method is used later, the DriverManager selects the appropriate driver class from all the loaded drivers.

Syntax —

```
Class.forName("DriverName");
```

E.g.

```
Class.forName("org.postgresql.Driver");
```

2. explicit Registration using registerDriver() method.

Syntax —

```
DriverManager.registerDriver(driver);
```

E.g.

```
Driver d = (Driver) Class.forName("org.postgresql.Driver");
```

```
DriverManager.registerDriver(d);
```

*

Page No.			
Date			

* Important Methods of DriverManager class.

- | Method | Purpose |
|---|--|
| 1. DriverManager.getDriver() | — Returns JDBC driver installed in system. |
| 2. DriverManager.getConnection(
String url, String user,
String password) | — Returns a connection to a database |
| 3. registerDriver() | |
| 4. getLoginTimeout() | |
| 5. setLoginTimeout() | |

* 2. Step 2 - Establish a connection -

After registering the driver, the next step is to set up a connection.

A connection represents a session with a specific database, SQL statements are executed within this session.

The DriverManager.getConnection() method is used to establish the connection.

This method takes three arguments: a JDBC URL, a database username and a password. It returns a connection object and throws java.sql.SQLException.

Syntax -

```
Connection cn = DriverManager.getConnection("JDBC URL", "user", "pass");
```

e.g.

```
connection cn = DriverManager.getConnection(
    "jdbc:postgresql://localhost/mydb",
    "Username", "password");
```

* Important Methods of Connection class

1. void close() → closes a connection

2. setAutoCommit(boolean) -
sets the status of commit

② ~~void~~ setAutoCommit(boolean) -

③ boolean getAutoCommit()

④ void commit()

⑤ void rollback() → undo effect of statement from last commit

Step-3

Create a Statement Object

→ Once you have created a connection you can use it to execute SQL statements. This is done using statement object.

There are three types of statements in JDBC.

1. Statement : This represents a general SQL statement without parameters. The method createStatement() creates a statement object.

Page No.	
Date	

b. Prepared Statement! — This represent a precompiled SQL statement, with or without parameters.

Prepared statements are used for SQL commands that need to be executed repeatedly.

The method `prepareStatement(String sql)` creates a prepared statement object.

Callable Statement — callable statement objects are used to execute SQL stored procedures. The method `prepareCall(String sql)` creates a callable statement object.

* Statement `st = con.createStatement()`

* String query = "update author set name = ? where id = ?";

PreparedStatement `st = con.prepareStatement(query)`

* CallableStatement `st = con.prepareCall("call show_authors")`

* Step 4 — execute queries —

An SQL statement can either be a query that returns a results or an statements operations that manipulates the database in some way.

Page No.	
Date	

1. Using statement —

The statement object can be used to execute a static SQL statement without parameters.

* Following methods of the statement interface are used to execute the SQL statement.

1. ResultSet executeQuery(String sql) —

This is used for statements that returns an output result.

e.g. Select query.

2. int executeUpdate(String sql) —

This is used for a statements that do not return an output.

Such as UPDATE, DELETE, CREATE TABLE, DROP TABLE, ALTER TABLE.

Statements. It returns the number of rows changed.

3. boolean execute(String sql) —

Executes a query and returns true if the query returns a result. Used when we don't know whether an SQL statement is going to return result.

If there is result it can be obtained using the `getResultSet()` method.

ResultSet rs = stmt.executeQuery("select * from emp");

int result = st.executeUpdate("update employee
set ename = ? where
id = ?");

boolean ans = st.executeUpdate("Drop
table employee");

2) Using PreparedStatement:

PreparedStatement are used for SQL commands that need to be executed repeatedly but ~~used~~ with different values. These are precompiled SQL statements with parameters.

For parameters the SQL statement in a PreparedStatement can contain placeholders which can be changed at runtime.

The placeholders ~~are~~ ^{are} represented by actual "?" in the SQL command.

e.g.

String sql = "UPDATE authors SET
SET Name = ?
where id = ?";

PreparedStatement ps = con.prepareStatement
(sql);

Before the SQL statement is executed, the placeholders have to be replaced by actual values.

This is done by calling a setXXX(int n, XXX, x) method.

Page No.	
Date	

e.g.

```
String sql = "update authors set
```

```
set name = ?
```

```
where id = ?";
```

```
PreparedStatement ps = con.prepareStatement(sql);
```

```
ps.setString(1, "abc");
```

```
ps.setInt(2, 101);
```

```
int result = ps.executeUpdate();
```

③ * Using Callable Statement -

The CallableStatement interface supports stored procedures.

The connection class has a `prepareCall()` method to create a `CallableStatement` object. It defines a standard syntax for accessing stored procedures.

- Stored procedures that doesn't return a result set and does not take parameters.

Syntax - { Call procedure_name }

```
CallableStatement cst = con.prepareCall("{ call  
disp_Student }");
```

2. The syntax for a stored procedure that doesn't return a result but takes parameters →

~~{ call procedure_name [(? [, ?] ...)] }~~
 { call procedure_name [(? [, ?] ...)] }

e.g.

callableStatement cst = cn.prepareStatement(" { call calc_interest (? , ?) } ");

3. The syntax for a stored procedure that returns a resultset is

{ ? = call.procedure_name [(? [, ?] ...)] }

* IN and OUT parameters

Passing in any IN parameter to values to a CallableStatement object is done using the setXXX() method. If the stored procedure returns OUT parameter the JDBC type of each OUT parameter must be registered before the CallableStatement object can be executed. This is necessary because some DBMS require the SQL type which the JDBC type represents.

Registering the JDBC type is done with the method `registerOutParameter()` method. Then after the statement has been executed, the `getXXX()` method can be used to retrieve **OUT** parameter values.

```
CallableStatement cst = (cn.prepareStatement(
    "{ call getTestData(?, ?, ?) }");
cst.registerOutParameter(1, java.sql.Types.INTEGER);
cst.registerOutParameter(2, java.sql.Types.DECIMAL,
    9);
ResultSet rs = cst.executeQuery();
int i = cst.getInt(1);
java.math.BigDecimal n = cst.getBigDecimal(2);
```

⑤ process The Result * : —

A Resultset contains the results of executing an SQL query. The data is stored in a resultset in a tabular manner with column headings and the corresponding values returned by a query.

To access these values there are `getXXX()` methods where XXX is a type of
e.g., `getString()`, `getInt()`, `getFloat()` etc.

There are two forms of the `getXXX()` method.

1. Using `columnName` `getXXX(String columnName)`
2. Using `columnNumber` `getXXX(int columnNumber)`

e.g. ~~`rs.getXXX("s_id");`~~
`rs.getInt(1);`
`rs.getInt("s_id");`

* Step 6- Close the Connection -

After all work is done. It is important to close the connection.

But before the connection is closed, close the `ResultSet` and the statement objects. The `close` method can be used for closing.

e.g. ~~`rs.close();`~~
~~`st.close();`~~
`cn.close();`

* program to demonstrate all steps of Database connection (JDBC connection)

→

```
import java.sql.*;

class StudentInfo {
    public static void main(String args[])
        throws Exception {
        try {
            Class.forName("org.postgresql.Driver");
```

```

class.forName("org.postgresql.Driver");
System.out.println("Driver is loaded");
Connection cn = DriverManager.getConnection(
    "jdbc:postgresql://localhost:5432/yourdb", "username", "password");

if (cn == null)
{
    System.out.println("Connection is unsuccessful");
}
else
{
    Statement st = cn.createStatement();
    st.execute
    PreparedStatement
    String sql = "select * from student";
    ResultSet rs = st.executeQuery(sql);

    while (rs.next())
    {
        System.out.println("Roll No: " + rs.getInt(1));
        System.out.println("Name: " + rs.getString(2));
        System.out.println("Age: " + rs.getInt(3));
        System.out.println("Percentage: " + rs.getFloat(4));
    }

    PreparedStatement ps = cn.prepareStatement(
        "insert into student values (2, ?, ?)");
    ps.setString(1, "Ravi");
    ps.setInt(2, 18);
    ps.setFloat(3, 85.5);
    ps.executeUpdate();
}

```

Page No.			
Date			

```

System.out.println ("Enter roll, name and per");
int ip = sc.nextInt();
String name = sc.next();
float p = sc.nextFloat();

ps.setInt(1, ip);
ps.setString(2, name);
ps.setFloat(3, p);

```

~~ResultSet rs = execute()~~

ps.executeUpdate();

∴ ~~ps~~ while

~~ResultSet rs = ps~~

statement q = cn.createStatement();

~~String q =~~

~~stmt~~

ResultSet rs = st.executeQuery("select * from student");

while (rs.next())

{

System.out.println("Roll" + rs.getInt(1));

System.out.println("Name" + rs.getString(2));

System.out.println("percentage" + rs.getFloat(3));

}

}

}

Page No.	
Date	

* Scrollable and Updatable ResultSet

* 1 -

The default type of ResultSet is Forward-Only and Read-Only, which means that we have can only traverse it in forward direction only and we cannot perform any modifications on the ResultSet object.

A ResultSet object can be controlled in two ways.

1. Scrollable - The ResultSet scroll type indicates how the cursor moves in the ResultSet i.e. whether in the forward direction only or in both direction.

2. Updatable - The ResultSet concurrency indicates whether it is possible to make changes to the database using the ResultSet object.

The scroll type and concurrency type can be set while creating the Statement Object.

Syntax -

```
Statement st = cn.createStatement(int scrollType, int concurrencyType);
```

* ResultSet Scroll Types -

1. ResultSet.TYPE_FORWARD_ONLY -

The ResultSet is scrollable non-scrollable. Its cursor moves forward only.

from first to last.

Page No.			
Date			

2. ResultSet.TYPE_SCROLL_SENSITIVE —

The ResultSet is scrollable its cursor can move forward or backward and can be moved to a particular row or to a row whose position is relative to its current position.

The ResultSet is sensitive to changes that are made to the database while it is open. If the underlying column values are modified the new values are visible thus providing a dynamic view of the underlying data.

3. ResultSet.TYPE_SCROLL_INSENSITIVE —

The result set is scrollable, its cursor can move forward or backward and can be moved to a particular row or to a row whose position is relative to its current position.

The result set does not show changes that are made to the underlying database while it is open.

* ResultSet Concurrency: —

1. ResultSet.CONCUR_READ_ONLY: —

This indicate that a result set does not show changes that are made to the underlying database while it is open.

Page No.	
Date	

* ResultSet Concurrency * : -

1. ResultSet.CONCUR_READ_ONLY : -

This indicates that a result set (and the underlying database) cannot be updated from the program. There is no limit to the number of users who can access the database. Thus this setting allows the highest level of concurrency.

2. ResultSet.CONCUR_UPDATABLE * : -

This allows a result set (and the underlying database) to be updated through the program.

This reduces the level of concurrency. Only one user is allowed to change the data at a time.

* e.g.

```
Statement stmt = Ch.createStatement(ResultSet.  
TYPE_SCROLL_SENSITIVE, ResultSet.  
CONCUR_UPDATABLE);
```

* Following are the scroll types and concurrency type.

1. ResultSet.TYPE_SCROLL_INSENSITIVE → The result set is a scrollable but not sensitive to database changes.

2. ResultSet.TYPE_FORWARD_ONLY — The result set is not scrollable.

Page No.			
Date			

3. ResultSet.TYPE_SCROLL_SENSITIVE -

The resultSet is scrollable and sensitive to database changes.

4. ResultSet.CONCUR_READ_ONLY -

The resultSet cannot be used to update the database.

5. ResultSet.CONCUR_UPDATABLE -

The resultSet can be used to update the database.

★

1. beforeFirst() - Default position. Puts cursor before 1st row of ResultSet.

2. first() - puts cursor on 1st row of ResultSet.

3. last() - puts cursor on last row of ResultSet.

4. afterLast() - puts cursor after (beyond) last row of ResultSet.

5. next() - to move to the next row in ResultSet.

6. previous() - to move to the previous row in ResultSet.

7. deleteRow() - Delete the current row from the resultSet and underlying database.

Page No.			
Date			

8. `getRow()` - Retrieves the current row number.
9. `insertRow()` - Inserts the contents of the insert row into the ResultSet Object and into the database.
10. `updateRow()` - Update the row in the underlying database with the new contents of the current row of this ResultSet Object.
11. `getXXX(String columnName)` -
12. `isFirst()` -
13. `isLast()` -
14. `updateXXX(int columnNumber, XXX value)` -
15. `updateXXX(String columnName, XXX value)` -

✓ Updating a Row —

→ The `updateRow()` or `updateXXX()` method takes two parameters: the first to indicate which column is to be updated and the second to give the value to assign to the specified column. The column can be specified by giving either its name or its index.

∴ e.g. `int n = rs.getInt(3); //`

```
rs.updateInt(3, 88);
```

```
int n = rs.getInt(3);  
updateRow();
```

OR

```
int n = rs.getInt("SCORES");
```

```
rs.updateInt("SCORES", 88);  
updateRow();
```

An application may explicitly cancel the update to a row by calling the method `cancelRowUpdates()`

e.g

```
Statement st = con.createStatement(ResultSet.  
TYPE_SCROLL_SENSITIVE, ResultSet.  
CONCUR_UPDATABLE);
```

```
String query = "select * from student";
```

```
ResultSet rs = st.executeQuery(query);  
while (rs.next())
```

```
{  
    if (rs.getInt(1) > 0)
```

```
{  
    double increase = ...
```

```
    double curper = rs.getDouble("perc");
```

```
    rs.updateDouble("perc", curper + increase);
```

```
    rs.updateRow();
```

```
}  
}  
}
```

* Inserting a Row -

New row may be inserted into a ResultSet table and into the underlying database table using JDBC API.

To make this possible, the API defines the concept of an insert row. This is a special row associated with the result set for building the row that is to be inserted.

Steps -

1. Call the ResultSet method `moveToInsertRow()` which positions the cursor on the insert row.
2. Call the appropriate `updateXXX` method to add column values in the insert row.
3. When all of the columns of the row to be inserted have been set, call the method `insertRow()`. This method adds the insert row to both the result set and the underlying database simultaneously.

e.g.

```
rs.moveToInsertRow();
rs.updateString(2, "xyz");
rs.updateDouble(3, 90.32);
rs.insertRow();
rs.first();
```

Page No.	
Date	

* Deleting Row.

The `deleteRow()` method deletes the current row from the result set and underlying database before calling `deleteRow()` the application must position the cursor on the row it wants to delete.

rs.first()

rs.deleteRow()

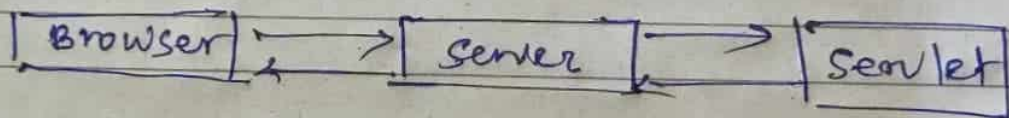
Servlet and Jsp -

Page No.	
Date	

- * A server is a computer that responds to request from a client. Typical client requests are to fetch a web page, upload files or download a file, send email etc. A client could be the browser or other slw making these request.

Servlets are small java programs that execute on the server side.

A java servlet is a platform independent application component that is hosted in a server.



Servlet operation -

- * Two packages make up the servlet architecture.

1. `javax.servlet` - The `javax.servlet` package contains the generic interfaces and classes that are implemented and extended by all servlets.

2. `javax.servlet.http` - The `javax.servlet.http` package contains all servlet classes that are HTTP protocol specific.

Page No.			
Date			

Advantages of Servlet -

1. Efficiency - In servlet each request is handled by a ~~high~~ light-weight java threads for n requests. There are n threads but the memory holds only one copy of the servlet class. Thus better use of resource.

2. Platform Independent